Compiling and executing C++ code [1]

Update Oct 10, 2025

C++ is a compiled language, meaning that the source code you write (using an editor of your choice) has to be translated (compiled) into a file that can be run by the computer. We refer to the latter file as an executable.

We recommend using the open source gnu C++ compiler which is installed on the Windows PCs in the university's computing clusters and is also available on Linux and on Macs. There are many "integrated development environments" (IDEs) that combine the process of writing source code, compiling, running and debugging on a single platform, using fancy graphical interfaces. You're welcome to use any of these, but here we show you a relatively low-tech approach that will get you started quickly (and is, in many ways, actually preferable to full-blown IDEs).

For the instructions below we assume that you're working on the university's Windows PCs or on your own Windows laptop. (If you're working on Linux you'll presumably be familiar with the command line anyway). Rather than doing the usual Windows thing of "clicking on things", we will use a command line interface.

Now let's get started:

1. Go to the "Getting started" module on the course's Canvas page and download the code `hello_world.cpp`. Your browser may ask you where you want to save the file to, or it may just place the file into the `Downloads` folder. We'll assume that it's ended up there!

2. Search for the command line interface ("Command prompt") by entering "cmd" into the search box (usually pre-populated with "Search") at the bottom of the screen on the standard Windows desktop. Click on "Open" in the window that pops up and offers you the "Command prompt".

3. This will open a terminal into which you can enter commands after the ">" prompt. Note that the prompt will also show you which directory (a "folder" in Windows terminology; we will generally use the term "directory") you're in, so if your username is `mbbssmh2`, it'll show `C:\Users\mbbssmh2>`.

4. Change into the OneDrive drive directory. This is a cloud-hosted directory that keeps your data alive between different sessions. There's a little problem in that currently (Oct 2025) some students (and staff!) have multiple directories with that name but only one is the right one. Only the cloud hosted one will properly save your data! Aaaaargh!

   Here's what you do:

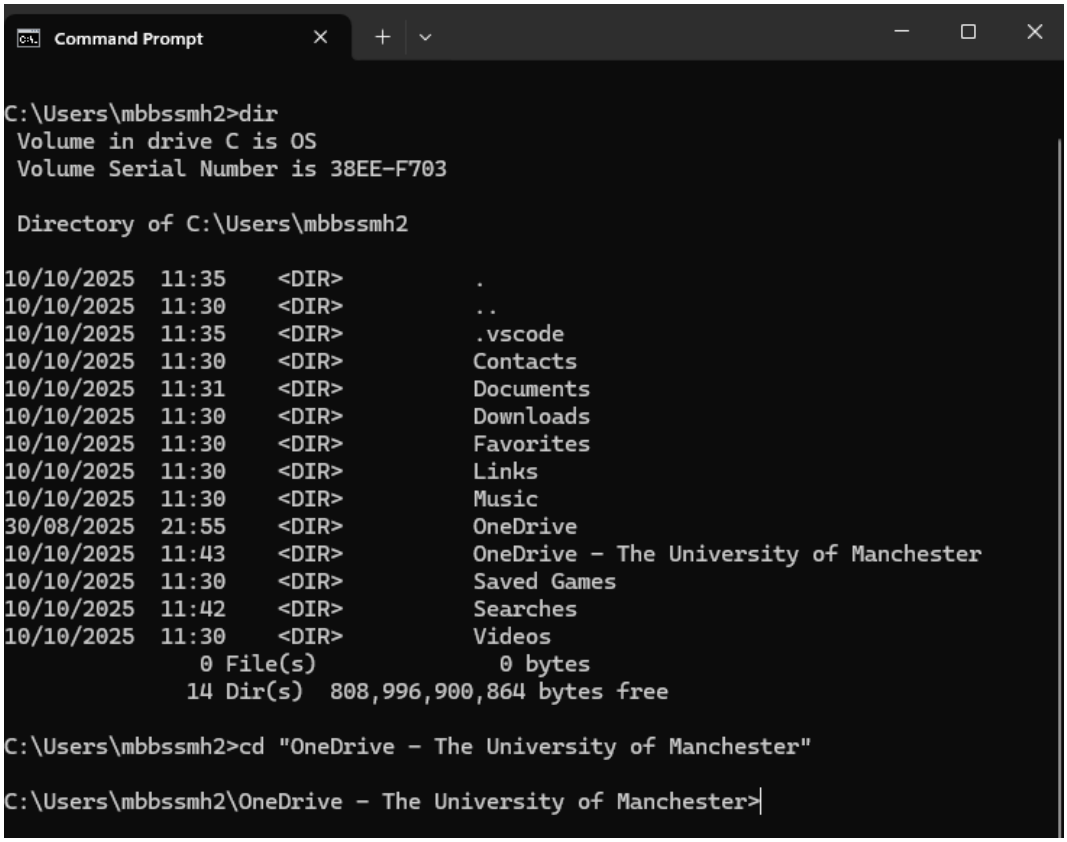
   - List the content of your home directory, using the `dir` command:

     ```
     dir
     ```

     This should list something like this:

[1] Version 2026/09/24.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`

2026/09/24, 09:49:00



Note that there are two `OneDrive` things. You want the one that has `"The University of Manchester"` in it. Unhelpfully, this name has spaces in it (not my fault; I have no idea who decided to set it up this way or why), so when you refer to it you have to quote it.

- Change into this directory by doing (`cd` = "Change Directory"):

```
cd "OneDrive - The University of Manchester"
```

as shown at the bottom of the screenshot. [**Note:** It gets worse! Sometimes this directory isn't mounted automatically, in which case it won't show up in that listing. If this happens, follow the instructions at the end of this document to mount it before continuing.]

5. Now that we're in this directory, let's create a new directory for the code that we develop in this course (`mkdir` = "Make Directory"):

```
mkdir ScientificComputing
```

6. Change into that directory

```
cd ScientificComputing
```

Note: You don't have to type out commands in full; if you hit the "Tab" key halfway through a command, the terminal will (cleverly!) try to complete the command for

2026/09/24, 09:49:00

you (if you've typed in enough information to make it unambiguous), so in this case typing "cd Sci" and hitting the "Tab" key should be all that's required.

7. Now let's create a new directory within the `ScientificComputing` directory to store the code for this warm-up exercise and again change into that newly-created directory:

```
mkdir WarmUpExercise
cd WarmUpExercise
```

8. Copy the downloaded code from the `Downloads` directory to your current directory

```
copy \Users\mbbssmh2\Downloads\hello_world.cpp
```

9. Have a look to see what's now in this directory, using the `dir` command which lists the contents of the current directory.

```
dir
```

10. Open the file in the Visual Studio editor:

```
code hello_world.cpp
```

You can now edit this file, and save any changes.

11. Now compile the code, using the gnu C++ compiler, g++, specifying the name of the executable with the -o flag

```
g++ -o hello_world hello_world.cpp
```

Note: Careful with the tab-completion here: if you use it too ambitiously/carelessly, you'll get `g++ -o hello_world.cpp hello_world.cpp` which would overwrite the source code with the executable.

Note: You can always edit the command line by moving along it, using the left/right arrow keys, and then adding/deleting what you don't like – here the ".cpp" in the auto-completed name of the executable.

12. Assuming you haven't broken the code while editing it, you should now have an executable called `hello_world` in your directory. Have a look to check:

```
dir
```

Note: Somewhat confusingly, windows attaches a postfix `.exe` to the executable. You can ignore this!

13. You can run the executable by entering its name on the command line

```
.\hello_world
```

Doing this, the words "Hello world" should appear in the terminal. Success!

Note: You could use the command

```
hello_world
```

implying tacitly that the executable is located in the current directory, or

```
hello_world.exe
```

They all do the same thing!

14. Now edit the code to introduce some deliberate errors, e.g. by deleting some of the semicolons at the end of the lines – they're important (don't worry, we'll tell you about this properly in the videos!). The code now won't compile any more and you'll get error messages that tell you which line the compiler gets confused about. Identify these lines in the editor and edit the code until it's working again.

    Note: When in the terminal you can use the up arrow to retrieve previously issued commands (and use the the up/down keys to navigate along the entire history of previous commands). This means that you don't have to type the same command over and over again. Once retrieved, you can also edit a command on the command line to adjust it.

15. Note that next time you come back to the computer you don't have to recreate the directory structure that you've just created, but once you're in the terminal you can change directly into the relevant directory by doing

    ```
    cd OneDrive\ScientificComputing\WarmUpProject
    ```

    and then continuing your work. It makes sense to maintain separate directories for different projects/exercise sheets, so if you want to put your work for Exercise Sheet 1 into a directory `Sheet1` in the `ScientificComputing` directory, do

    ```
    cd \Users\mbbssmh2\OneDrive\ScientificComputing
    mkdir Sheet1
    cd Sheet1
    ```

16. Note that if you open the Visual Studio editor without specifying a filename,

    ```
    code
    ```

    or with the name of a file that doesn't exist yet,

    ```
    code new_file.cpp
    ```

    you may get prompted to specify type of the document. Choose C++. Visual Studio may also offer to install various additional extensions. It's up to you if you wish to use them.

17. Finally, while we will discuss later in the course how to write results to specific files, you can always write (redirect) output that appears on screen into a file by using the `>` operator. So if

    ```
    .\hello_world
    ```

    makes the text "Hello world" appear on the screen, then

    ```
    .\hello_world > output.txt
    ```


2026/09/24, 09:49:00

will write "Hello world" into the file `output.txt`. To quickly see what's in that file (without having to open it in an editor like Visual Studio), you can use the `more` command:

```
more output.txt
```

which will list the contents of the specified file on the screen.

18. Other useful commands are:

    - `move` which renames files and/or moves them to a different directory:

      ```
      # This renames the file locally
      move original_file.dat renamed_file.dat

      # This moves the file to the directory above the current one
      move file.dat ..\

      # This moves the file to the directory two levels above
      # the current one and renames it at the same time
      move file.dat ..\..\moved_file
      ```

    - `del` which deletes files

      ```
      del rubbish_file_that_clutters_up_my_filespace.txt
      ```